

第9课 软件架构— 行为模式

北京交通大学软件学院

严禁复制





01 复制 基础的单体架构模式

单体架构模式-part1

1. Data flow模式
2. 任务调度模式
3. 交互模式
4. 仓储模式
5. 远程调用模式

严禁复制

01 Data flow

- Batch批处理
- Pipes & Filters管道和过滤器
 - 统一接口的管道和过滤器

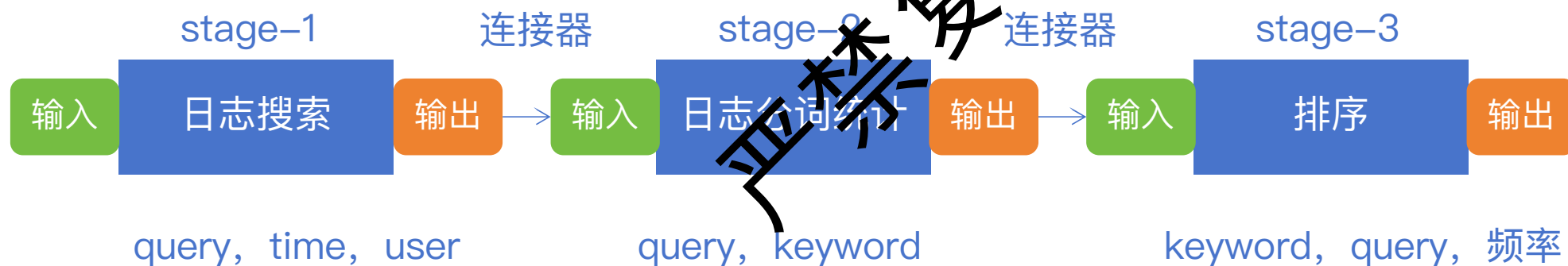
数据在系统中如何流动、转换和处理。

其核心思想是将系统分解为一系列处理单元（例如过滤器、处理器、函数等），这些单元通过数据流连接起来，形成一个完整的数据处理流程。

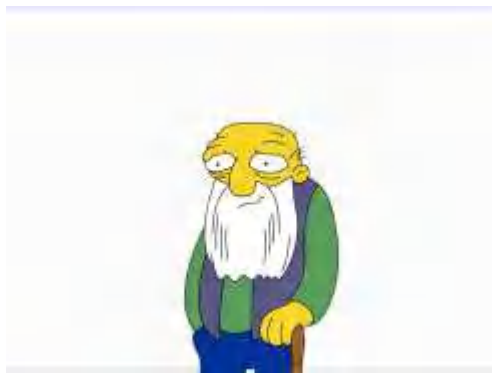
严禁复制

1-1 批处理模式

- 系统分为多个独立的模块，他们顺序执行；
- 前一个模块的输出是后一个模块的输入；
- 案例：100G条搜索日志找出最高频的10个词



批处理加工模式是最古老的架构模式！



1-1 批处理模式

- 要素：
 - 独立的可执行程序
- 约束：
 - 一个stage的输出连接到下一个stage的输入（不能给其它人）
 - 必须等待前一个完成后才能执行下一个

严禁复制

1-1 批处理模式

01

优势

组件之间松耦合
可配置
容易debug

02

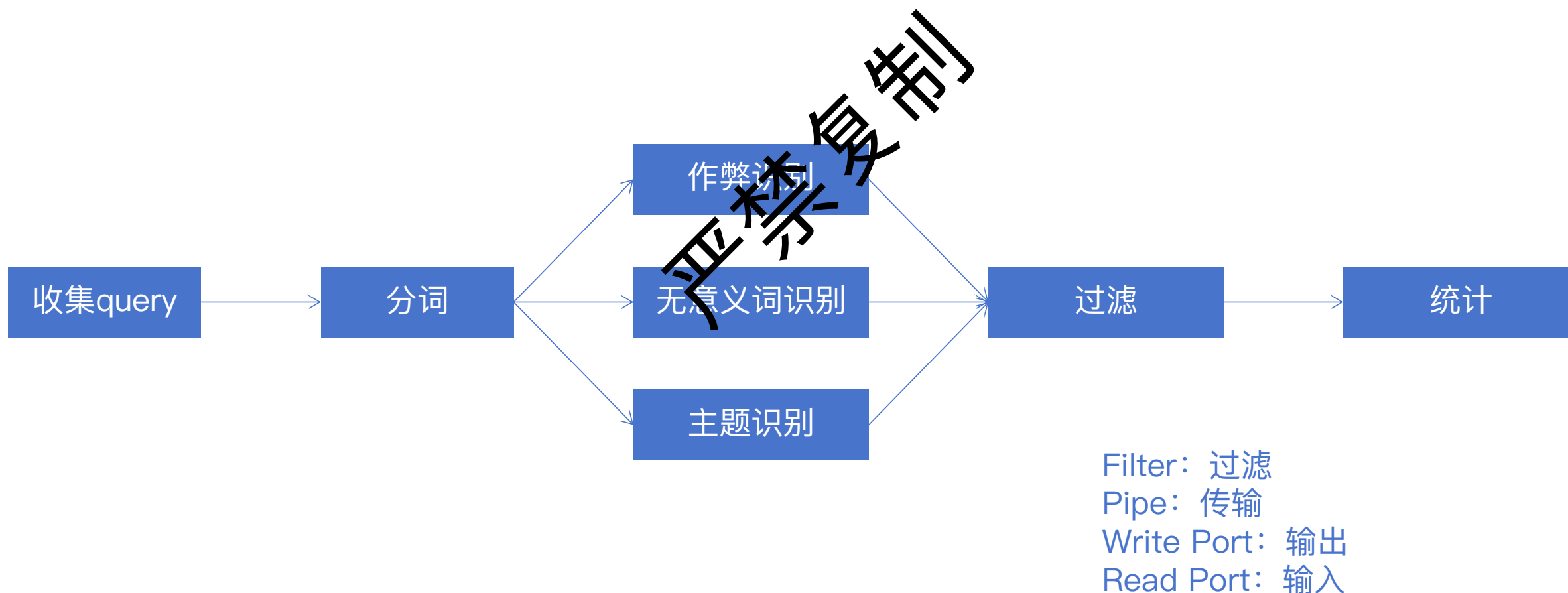
挑战

不支持交互（中途变更）
需要外部系统协调（管理依赖关系）
不支持并发
延迟比较高

严禁复制

1-2 管道和过滤器

- 数据通过管道（pipe）流动，通过过滤器（filters）处理



1-2 管道和过滤器

- Filter：用来对data做处理的组件
 - 可以并行执行
 - 类型：
 - data source（整个系统的输入）
 - flow（处理环节）
 - sinks（整个系统的输出）
- Pipe：将一个filter的内容输出给另外一个filter
 - Buffer size
 - Data format
 - 交互协议（开始，结束）

1-2 管道和过滤器

- 约束

- 一个管道只能有1个出口和入口
 - 将一个filter 的output连接到另一个filter的input
- filter必须实现他们的exchange format
 - 从一种格式变成另一种（音频to文本）

严禁复制

1-2 管道和过滤器



优势

- 容易理解系统整体
 - 整体行为 = 每个过滤器的行为之和
- 可复用性
 - filter可以被重新组合
- 进化和扩展
 - 创建新的filter
 - 替换旧的filter
- 可测试：可以独立测试每个filter
- 性能：允许各个filter的并行执行

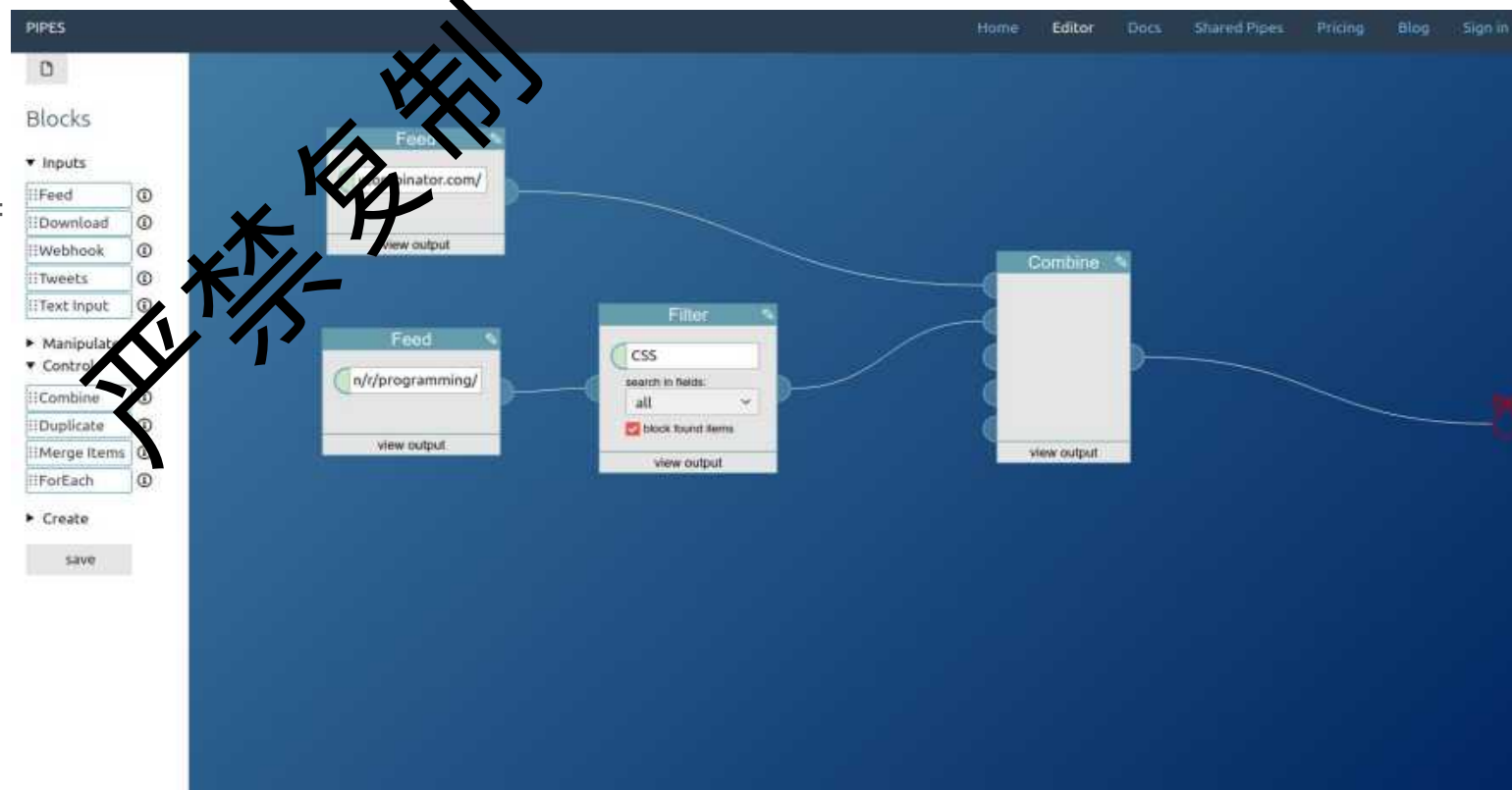


挑战

- pipes可能造成delay
- 难以传递复杂的数据结构
- 不能进行交互：适应环境变化
- 溢出：后面的filter收到太多数据

1-2 管道和过滤器-例子

- unix: `cat /var/log/syslog | grep "error" | wc -l`
- Yahoo Pipes
- Java Streams
 - `list.stream().filter(x -> x % 2 == 0) // 保留偶数`
- Flow based programming
 - 传统编程：控制流+服务。
 - Flow-Based Programming:
 - 节点+连接，无中心控制。
- Stream programming
 - Flink, Storm



1-3 管道和过滤器-统一接口

- 这是一种pipe&Filter的变体，但是所有filter都使用统一的接口
- 要素：
 - 和Pipe&Filter一样： pipe, filter
- 约束：
 - 所有filter都使用统一的接口

严禁复制

1-3 管道和过滤器-统一接口

- 优势

- 每个filter独立开发
- 每个filter可配置
- 系统理解更容易（不需要理解接口）

- 挑战

- 性能问题：将不同数据格式转化成统一接口
 - JSON序列化

严禁复制

1-3 Pipes & Filters-统一接口

- 例子

- Unix:
 - 所有命令都使用一个输入: `stdin`
 - 2个输出: `stdout`和`stderr`
- Web架构: REST
 - request:
 - HTTP 方法 + URI + Headers + Body (可选)
 - response
 - Response: 状态码 + Headers + Body (可选)

请求示例

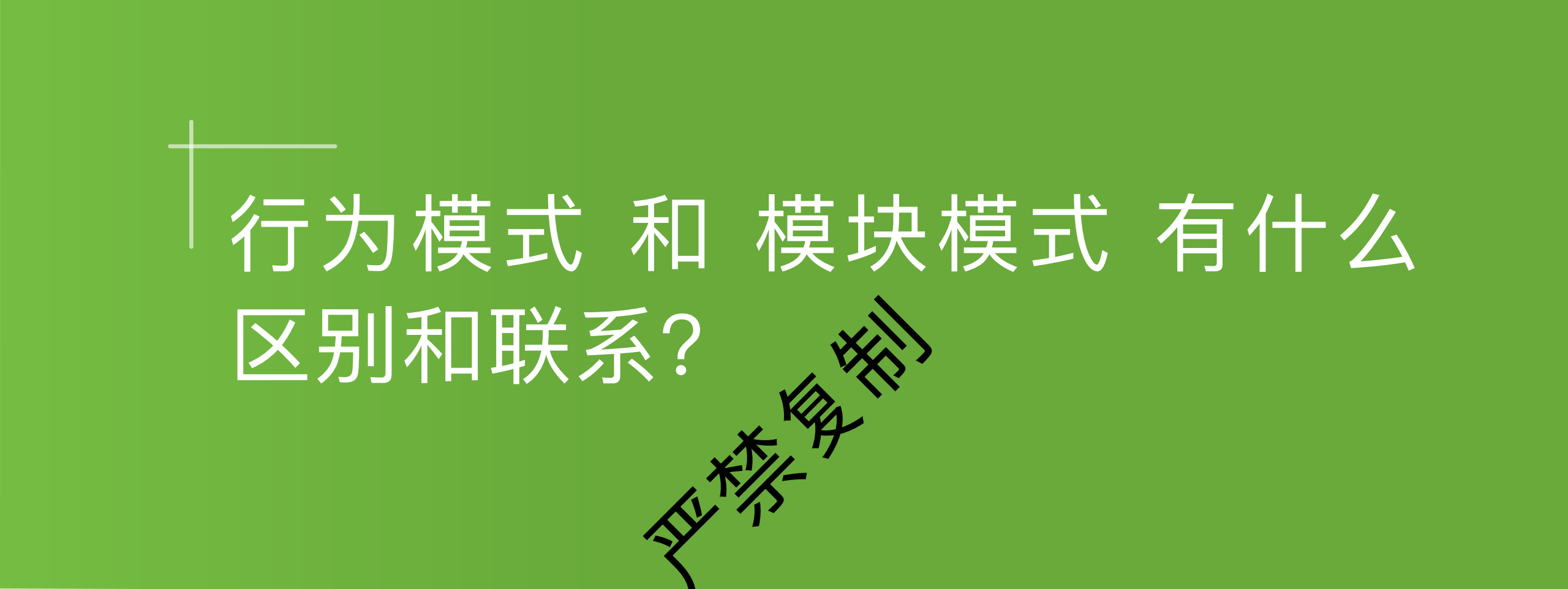
```
POST /users HTTP/1.1
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I...
```

```
{
  "name": "Bob",
  "email": "bob@example.com"
}
```

响应示例

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /users/456

{
  "id": 456,
  "name": "Bob",
  "email": "bob@example.com",
  "links": [
    { "rel": "self", "href": "/users/456" }
  ]
}
```



行为模式 和 模块模式 有什么
区别和联系？

严禁复制

思考题

2 任务调度

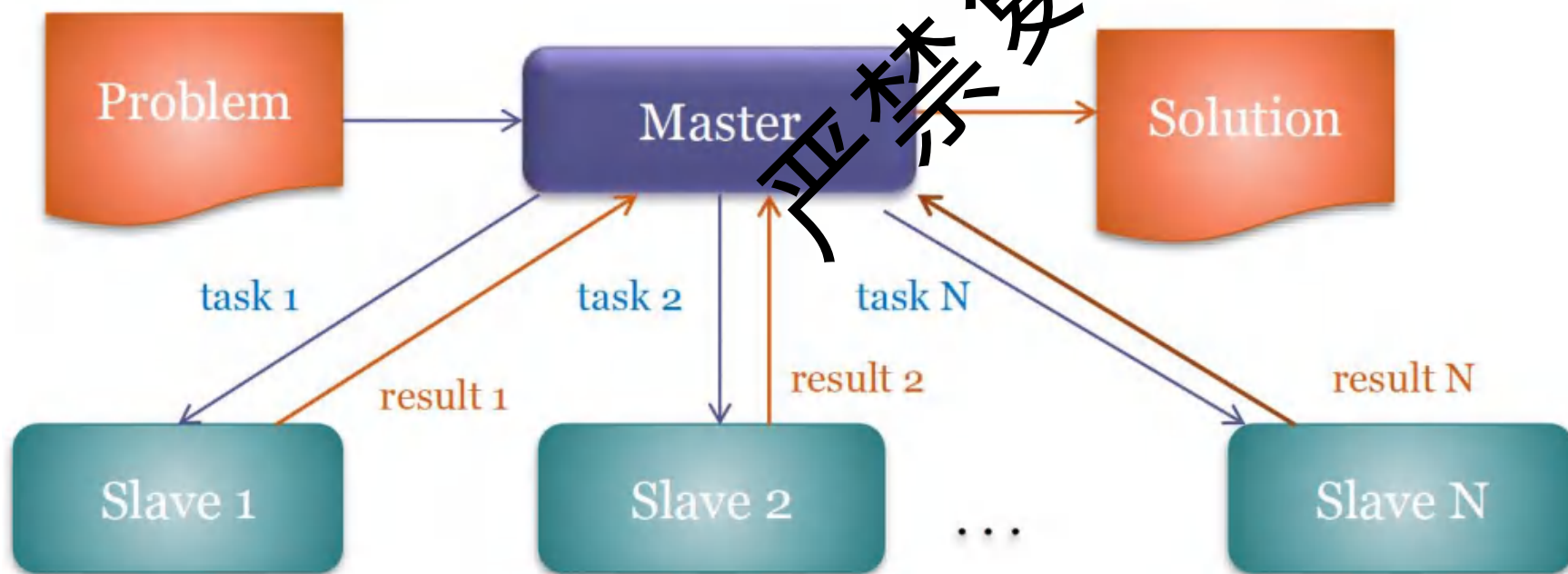
- Master-Slave

巨大的任务，很多的计算机；
如何才能高效利用资源，避免浪费和阻塞；
具备扩展性，提升系统容量；

严禁复制

2-1 Master-Slave

- Master把任务分拆成子任务
- 把每个子任务分配给不同的node
- 将子节点的结果组合成最终结果



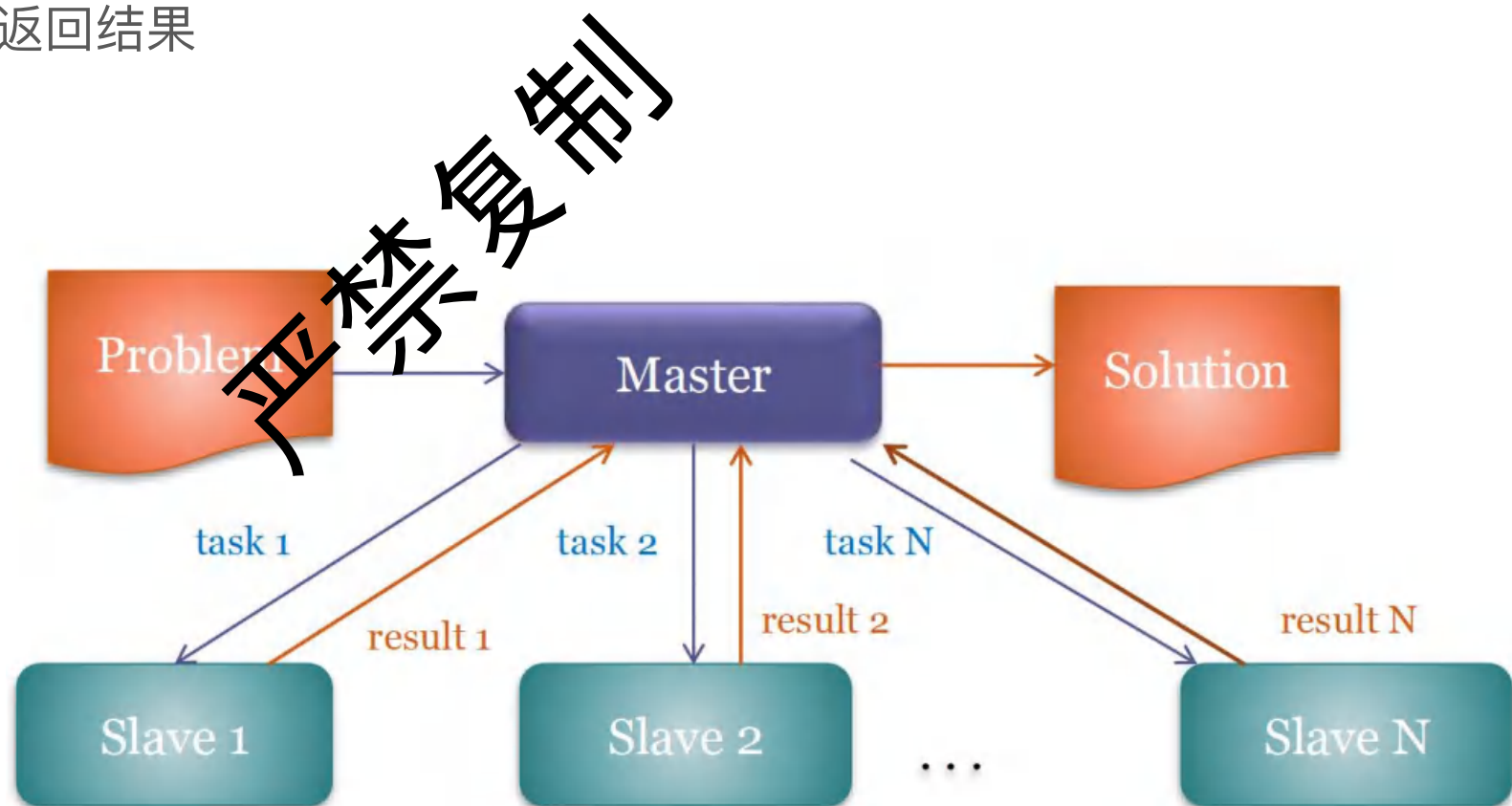
2-1 Master-Slave

- 要素

- master: 协调任务执行
- slave: 执行一个任务并返回结果

- 约束

- slave node只负责计算
- master负责控制



Master-Slave

01

优势：

并行计算

容错（slave失败了可以重新发起）

02

挑战

- salve协调的困难
 - 负载均衡
 - 可靠通信
 - 状态管理
 - 水平扩展
- master节点的可靠性
- 系统的配置

严禁复制

Master-Slave的应用

- 流程控制系统

- 监控制造流程，slave负责收集传感器信息

- 嵌入式系统

- 运行在设备中，slave负责传感器，master负责整体决策

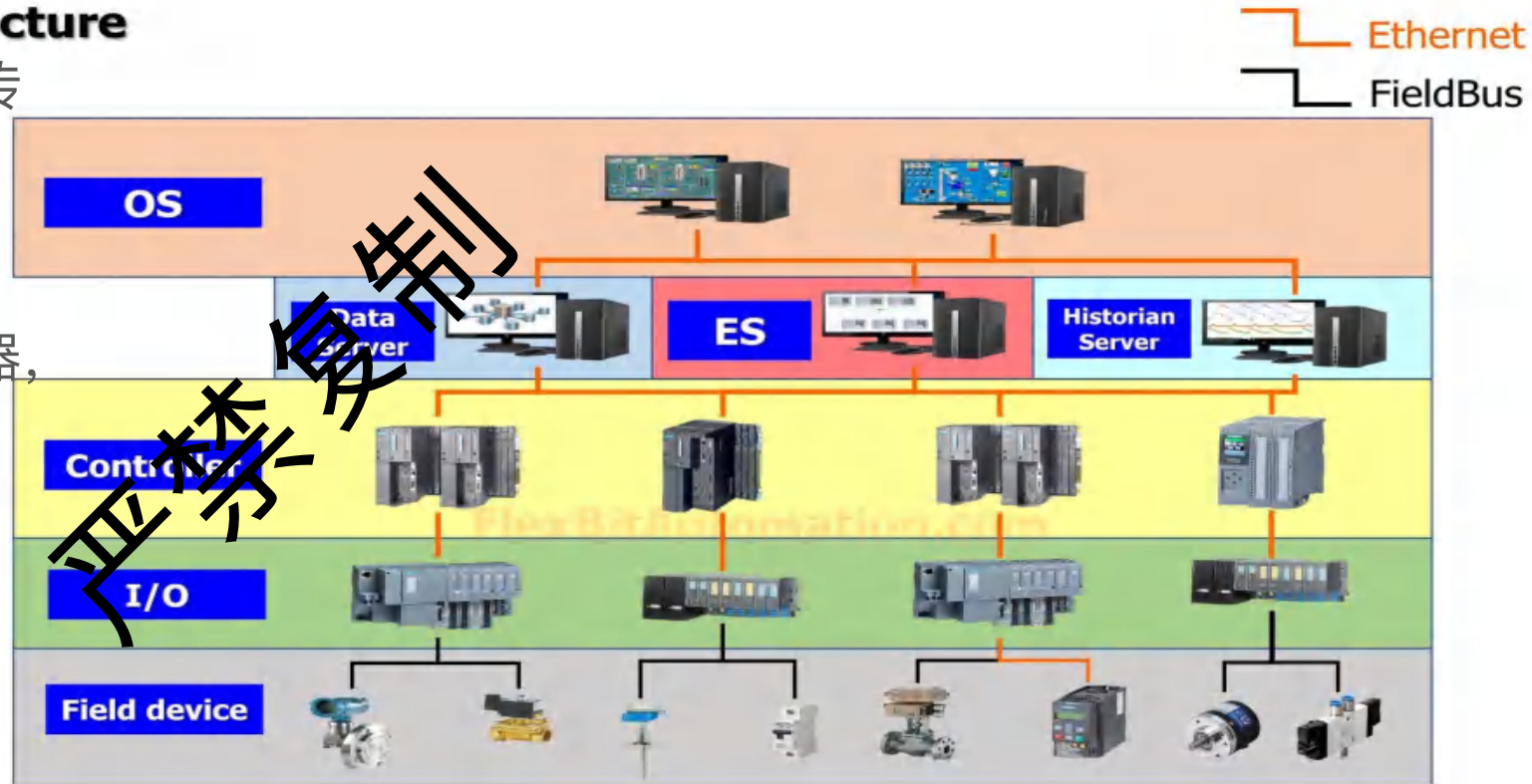
- 容错系统

- slave负责检查每个实例的状态

- 搜索系统

- 让slave执行搜索任务
- master汇总搜索结果

Structure



3-交互型系统

- MVC: Model-view-controller
- PAC: presentation-Abstraction-Control

实现人机交互;
解除模块耦合;
提升代码的可复用性;

严禁复制

不使用模式的UI系统

姓名

加载

```
// Database Connection
Class.forName("org.h2.Driver");
Connection connection = DriverManager.getConnection("jdbc:
```

```
// Query Database
String query = "SELECT * FROM users WHERE name = ?";
PreparedStatement preparedStatement = connection.prepareStatement(query);
preparedStatement.setString(1, name);
ResultSet resultSet = preparedStatement.executeQuery();
```

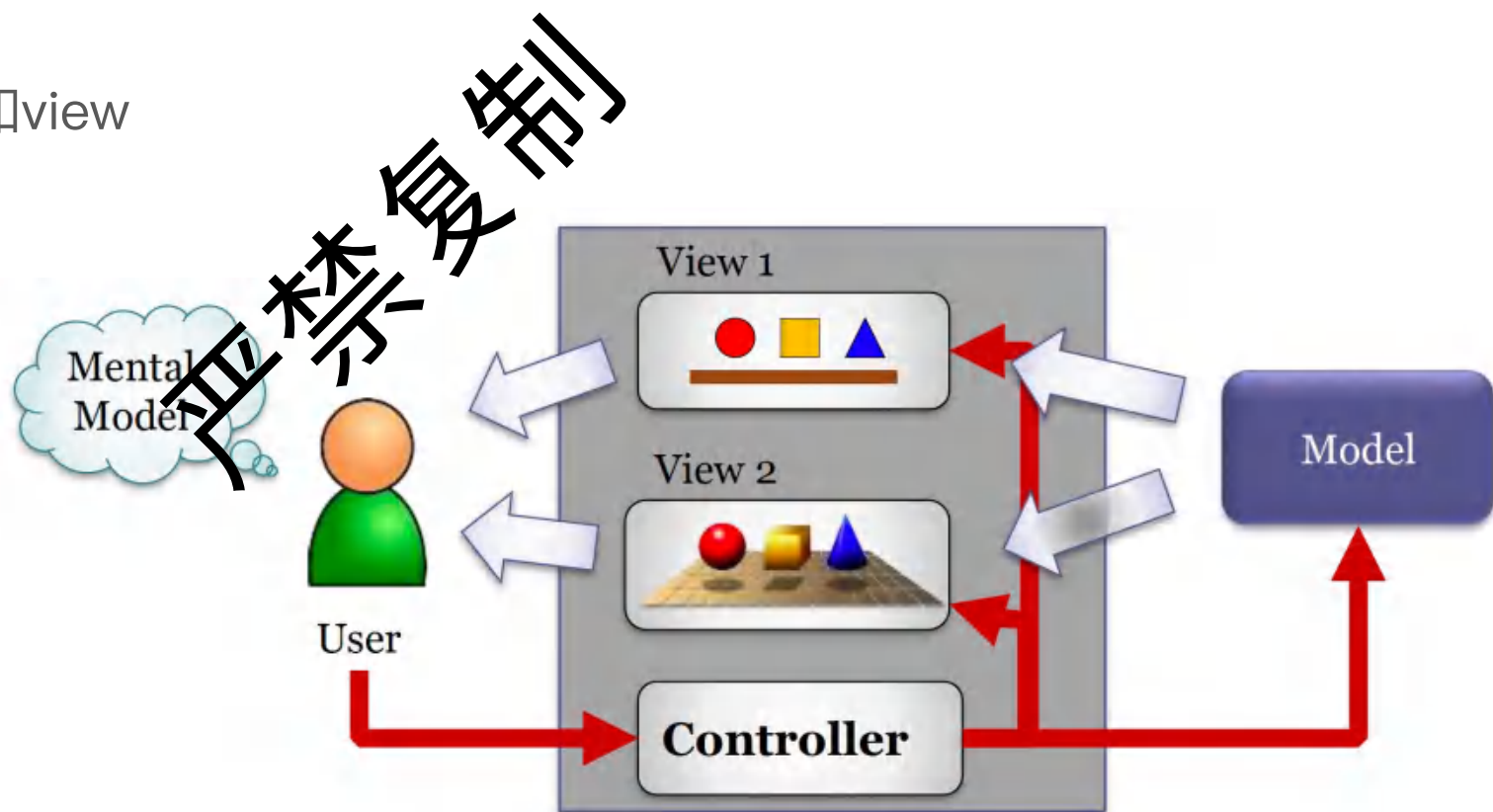
```
// Process Data and Build UI
StringBuilder data = new StringBuilder();
if (resultSet.next()) {
    int id = resultSet.getInt("id");
    String email = resultSet.getString("email");
```

```
// Simple Data Processing (Concatenation)
String processedData = "ID: " + id + ", Name: " + name;
data.append(processedData).append("\n");
```

```
// Update UI Directly
dataArea.setText(data.toString());
} else {
    dataArea.setText("No data found for name: " + name);
}
```

3-1 MVC – 基本定义

- MVC: Model–View–Controller
 - Trygve Reenskaug 于1979年提出
 - GUI
 - 使用controller分离model和view
 - view展示心智模型



MVC – 组成要素

- 要素

- Model: 业务逻辑, 业务状态
- View: 将状态展示给用户 (以合适的形式)
- Controller: 协调用户动作 (输入), model (改变), view (改变)

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>User Information</title>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
</head>
<body>
  <h1>User Information</h1>
  <p>Name: <span th:text="${userName}"></span>
  <p>Age: <span th:text="${userAge}"></span>
</body>
</html>
```

```
@Controller
public class UserController {

    @GetMapping("/user")
    public String getUserInfo(Model model) {
        // 模拟从数据库获取用户信息
        String name = "李四";
        int age = 25;

        // 将数据放入 Model (相当于 Value Stack)
        model.addAttribute("userName", name);
        model.addAttribute("userAge", age);

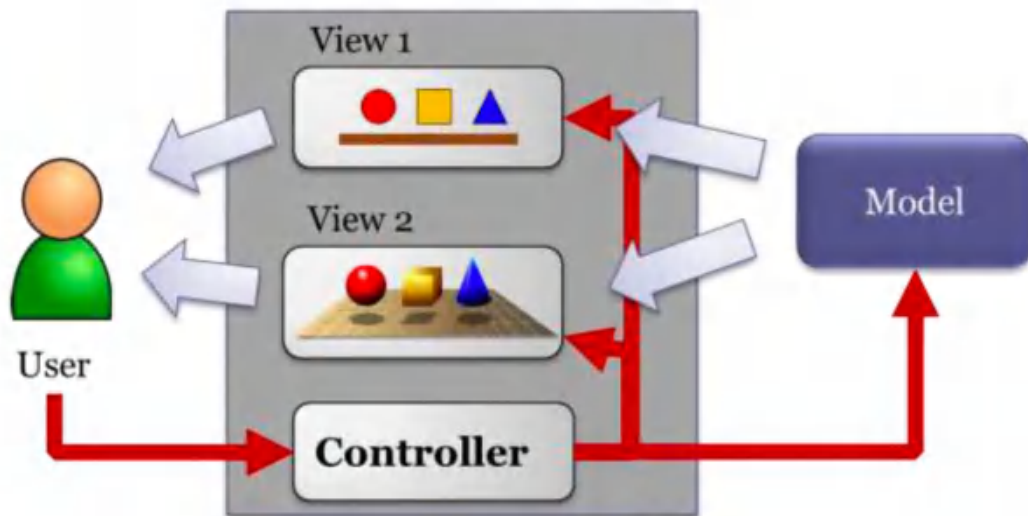
        // 返回 View 的名称
        return "user_info";
    }
}
```



MVC 约束

- Controller处理用户事件
 - 创建view
 - 删除view
 - 处理用户互动请求
- views只负责展示（接受用户输入）
- Models独立于controllers和views

禁止转载



MVC的优势和挑战



优势

- 一个Model支持多个view
- 多个view的同步更新
- 关注点分离：
 - 交互 (controller) state (model)
- 扩展性：
 - 新view、控制器、Look & feel修改
- 创建通用框架

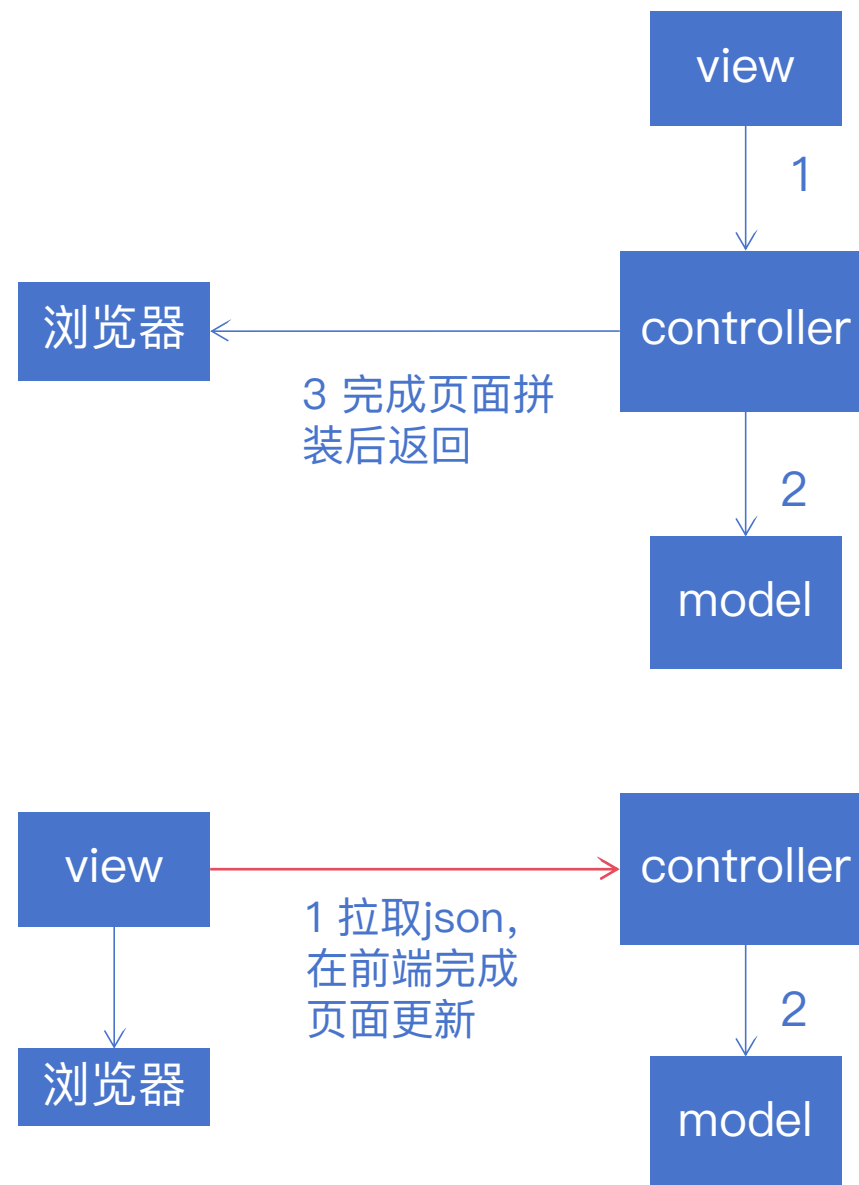


挑战

- 增加了GUI开发的复杂度
- controller和view之间存在耦合
- controller和view耦合model
- GUI工具

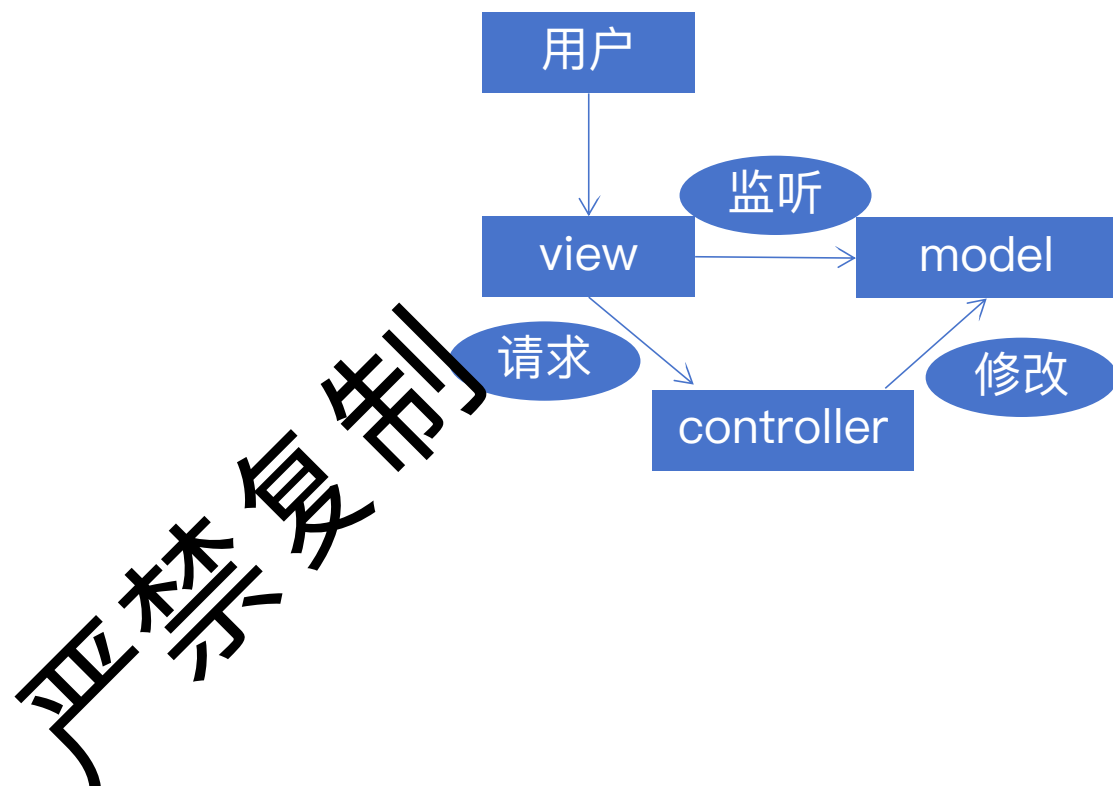
MVC应用

- web frameworks
 - Ruby on Rails, Spring MVC, Play,
- 变体
 - Push: controllers把指令发给views
 - Ruby on Rails, Struts1
 - Pull: controllers接收views传来的指令
 - Play framework, Struts2



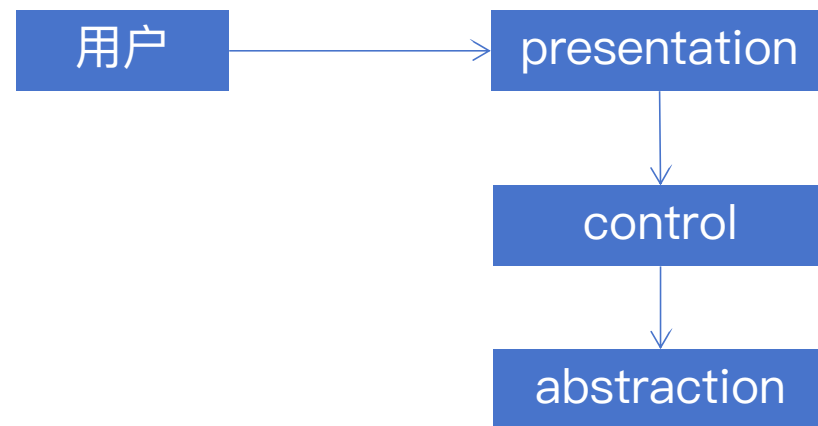
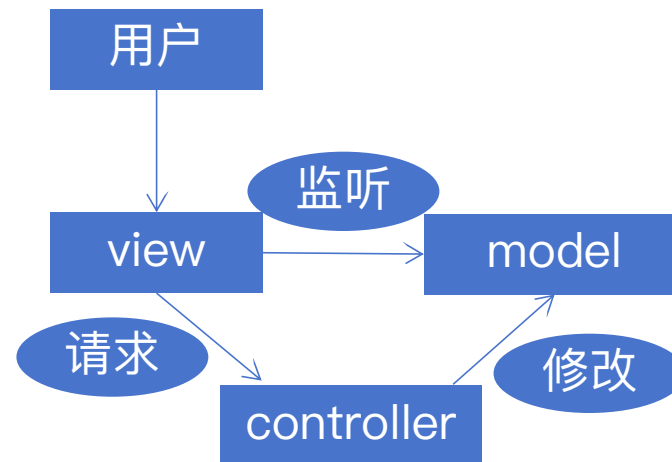
MVC的变体

- PAC
- Model–View–Presenter
 - presenter把结果更新到view
- Model View ViewModel
- Model View Update



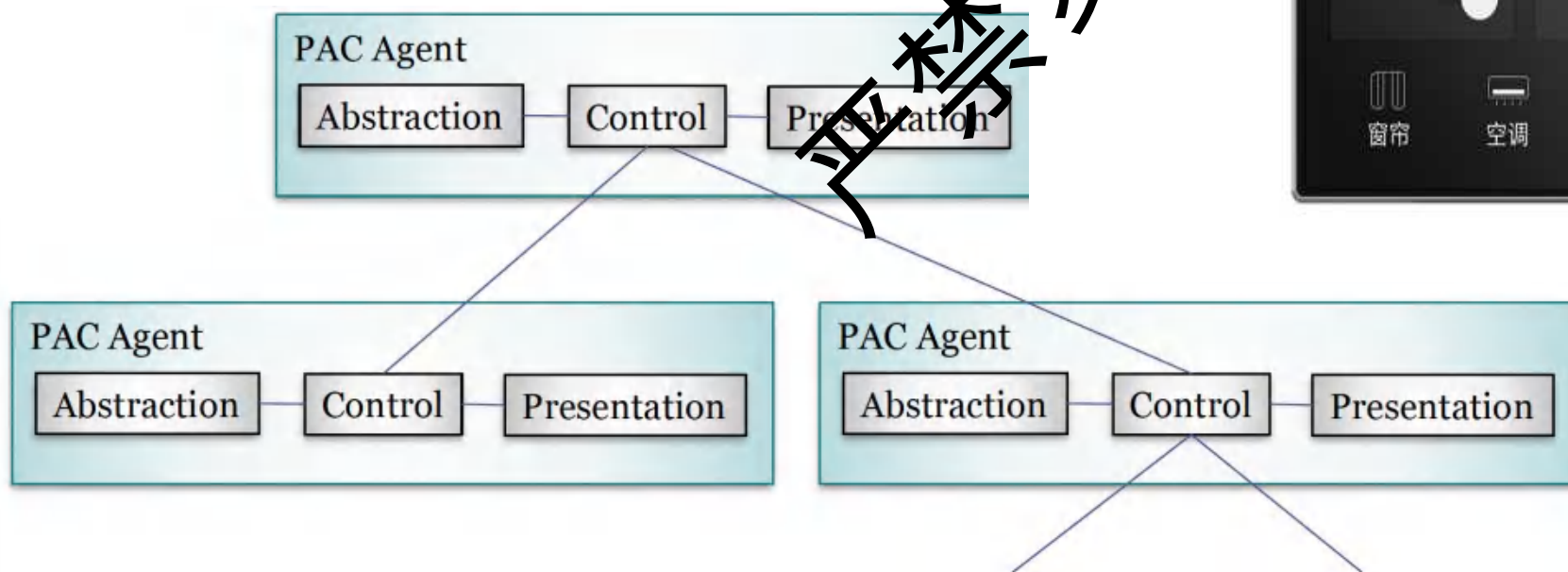
PAC

- Presentation：负责用户界面的显示和用户交互。
 - 它接收用户的输入，并将其传递给 Control。
 - 同时，它接收来自 Control 的数据，并将其显示给用户。
- Abstraction：负责数据的管理和业务逻辑的实现。
 - 它接收来自 Control 的请求，并根据请求访问和操作数据。
 - Abstraction 关注的是数据的存储、检索、处理和验证。
- Control：负责协调 Presentation 和 Abstraction 之间的交互。



PAC

- 每个PAC是单元是一个Agent
- 顶层Agent控制下层Agent
- 每个Agent保持独立，是可替换的模块



PAC的要素

- 要素

Agents

- 视觉方面
- 抽象模型
- 抽象控制

agent的层次结构

- 约束

每个agent控制某方面的功能

每个agent的abstraction和presentation不互通

所有通信都通过control

严禁复制

PAC应用的例子

- 应用
 - 网络监控系统（设备=agent，共享监控逻辑）
 - 无人机集群
 - Drupal：开源的内容管理系统
- 与MVC的关系
 - 层次化的MVC



PAC

1

优势

关注点分离：agent分开

支持修改和扩展：修改一个agent不影响其它

多任务：不同agent可以运行在不同的线程或者机器上

2

挑战

系统复杂性：太多agent难以维护

控制器的复杂性：

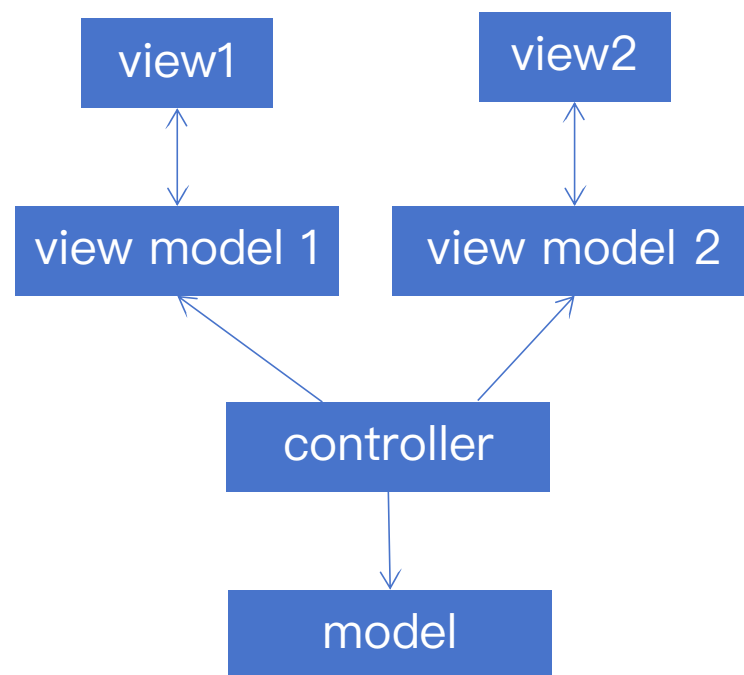
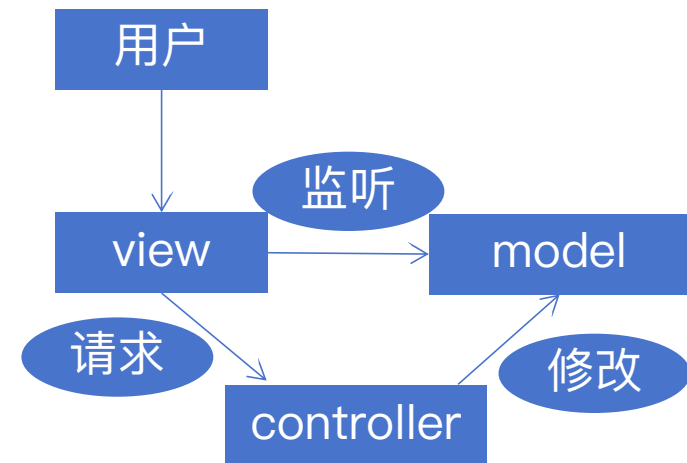
通信复杂对control造成压力

性能

agent之间也会互相通信

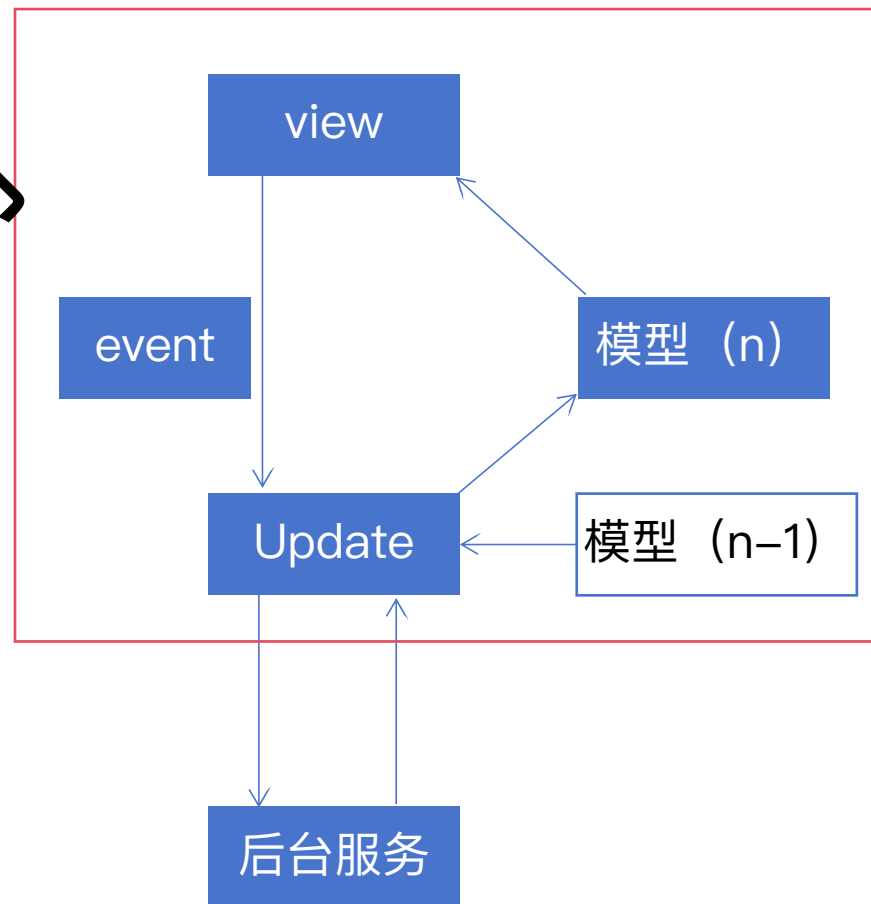
Model View ViewModel

- Model (模型): 数据和业务逻辑。
 - Model 它与 MVC 中的 Model 概念类似。
- View (视图): 用户界面的可视化表示。
 - 负责显示数据, 并接收用户的输入。
 - 应该尽可能地简单, 只包含 UI 相关的逻辑。
- ViewModel (视图模型): View 的抽象。
 - 从 Model 获取数据, 转换给view。
 - 处理用户的输入, 给 Model。
 - 不应该包含任何 UI 相关的逻辑。



Model view update

- Model (状态): Model 是应用程序的状态。
 - Model 不可变, 修改时必须创建一个新的 Model 副本;
- View (显示): View 是 Model 的可视化表示。
 - View 根据当前的 Model 状态进行渲染;
 - 它接受 Model 作为输入, 并返回用户界面;
- Update (更新): Update 是一个函数
 - Model + Message = 新 Model。
- Message (消息/事件): 用户交互或其他事件
 - Message 描述了发生了什么事情, 例如按钮被点击。



思考题



这个聊天网页，应该用什么模式：

A MVC

B PAC

C MVM

D MVV

严禁复制

4-Repository

- shared data 共享数据
- Blackboard 黑板
- Rule based 规则驱动

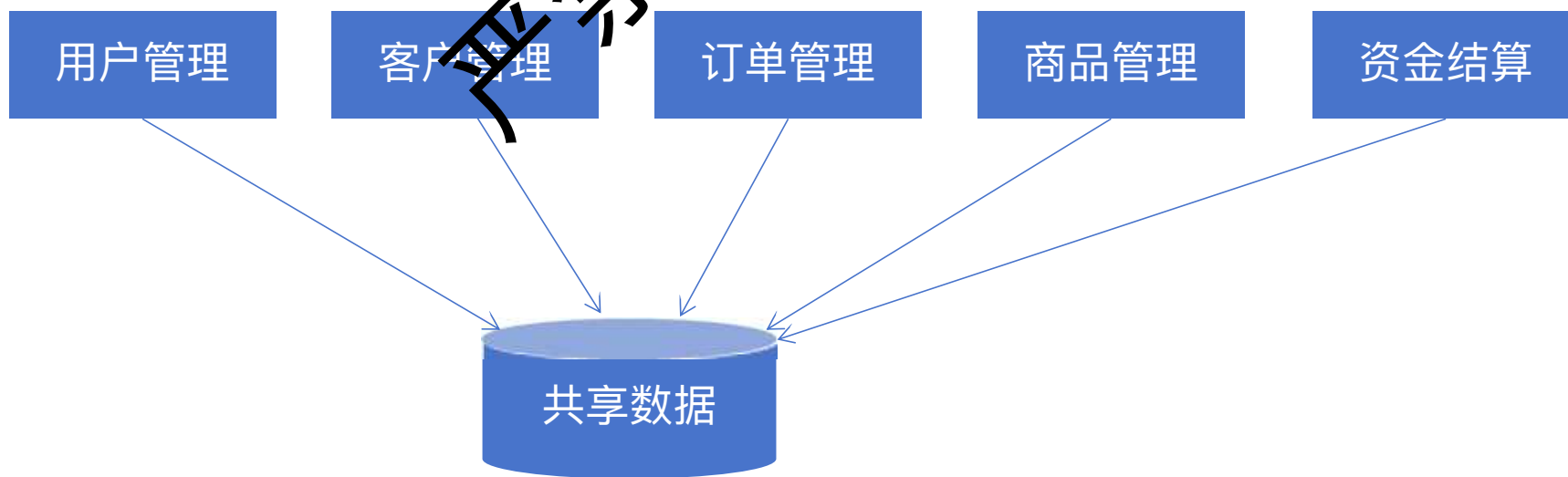
如何有效地组织、管理和利用数据，以实现特定的目标。

它们关注数据的存储、访问、共享、转换和利用，以及如何基于数据进行决策和推理。

严禁复制

共享数据

- 独立组件共同访问同一个数据存储对象
 - 使用集中式数据存储的应用系统
- 要素
 - 数据库或者共享存储库
 - 业务组件



共享数据

- 约束
 - 组件访问或修改的是系统的全局状态（非私有状态）
 - 组件之间不互通（rpc或event）
 - 只通过共享数据
 - 共享数据确保数据的稳定性和一致性
 - 唯一来源，没有私有状态

严禁复制

共享数据

1

优势

1. 组件之间互相独立
2. 数据永远一致
3. 简单，易于开发

2

挑战

1. 单点失败
2. 性能瓶颈
3. 并发访问冲突
4. 数据对象无限扩张
5. 模块包含潜在耦合

严禁复制

共享数据的应用

> 应用例子

很多系统采用

新功能=加数据+加代码+加页面

加在哪里：anywhere

变体

共享内存，仓储模式

Blackboard

Rule based system

严禁复制

Blackboard黑板

- 复杂问题难以解决
 - 每个组件是一个知识来源
 - 每个组件解决部分问题
 - 各个组件把部分问题的解决方案汇聚到Blackboard

严禁复制

Blackboard黑板模式

01

黑板模块：

维护中心数据结构
集成多个不完整答案
生成最终答案

02

知识模块：

解决部分问题
提交不完整答案

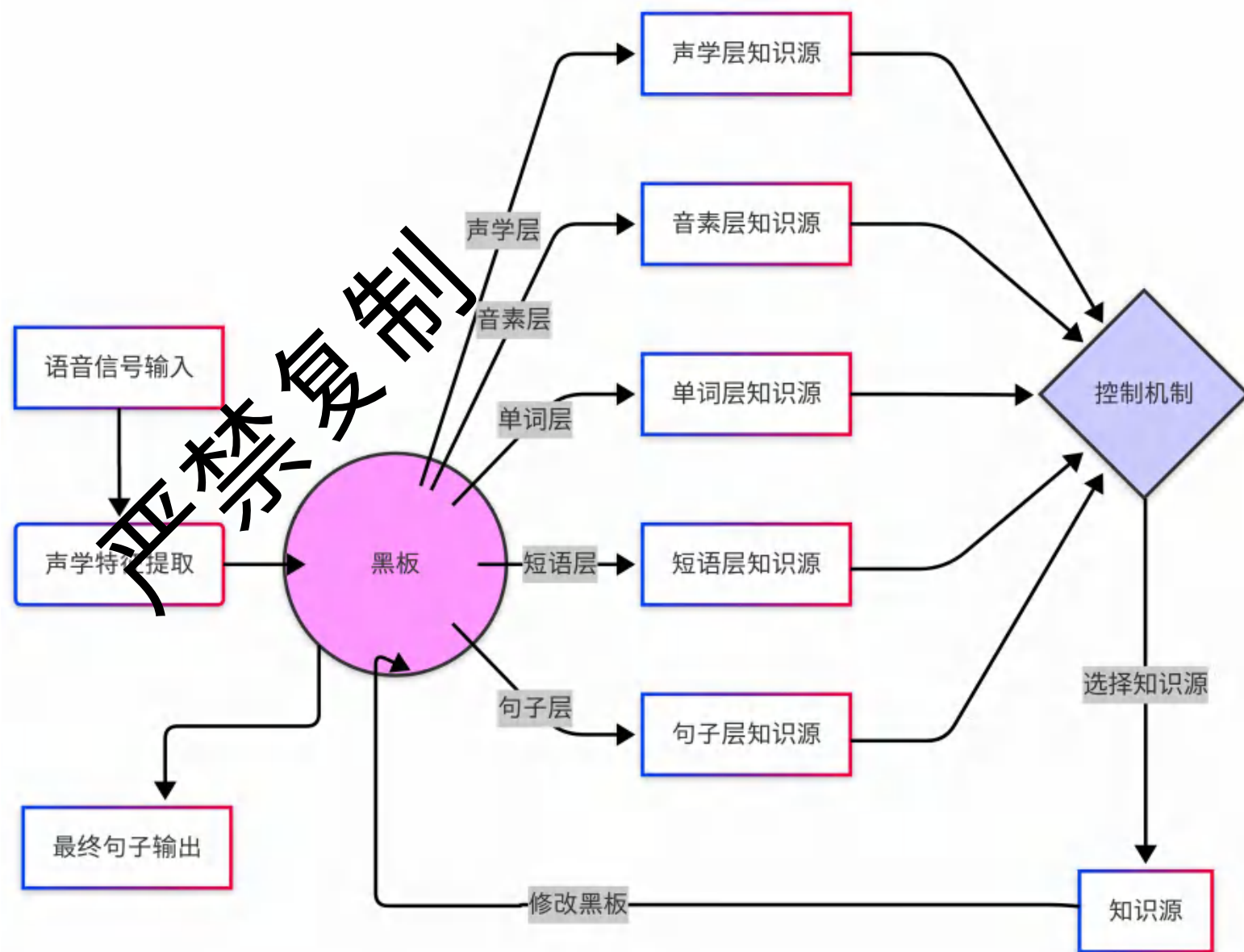
03

控制模块：

管理子任务
检查任务状态

Blackboard的应用

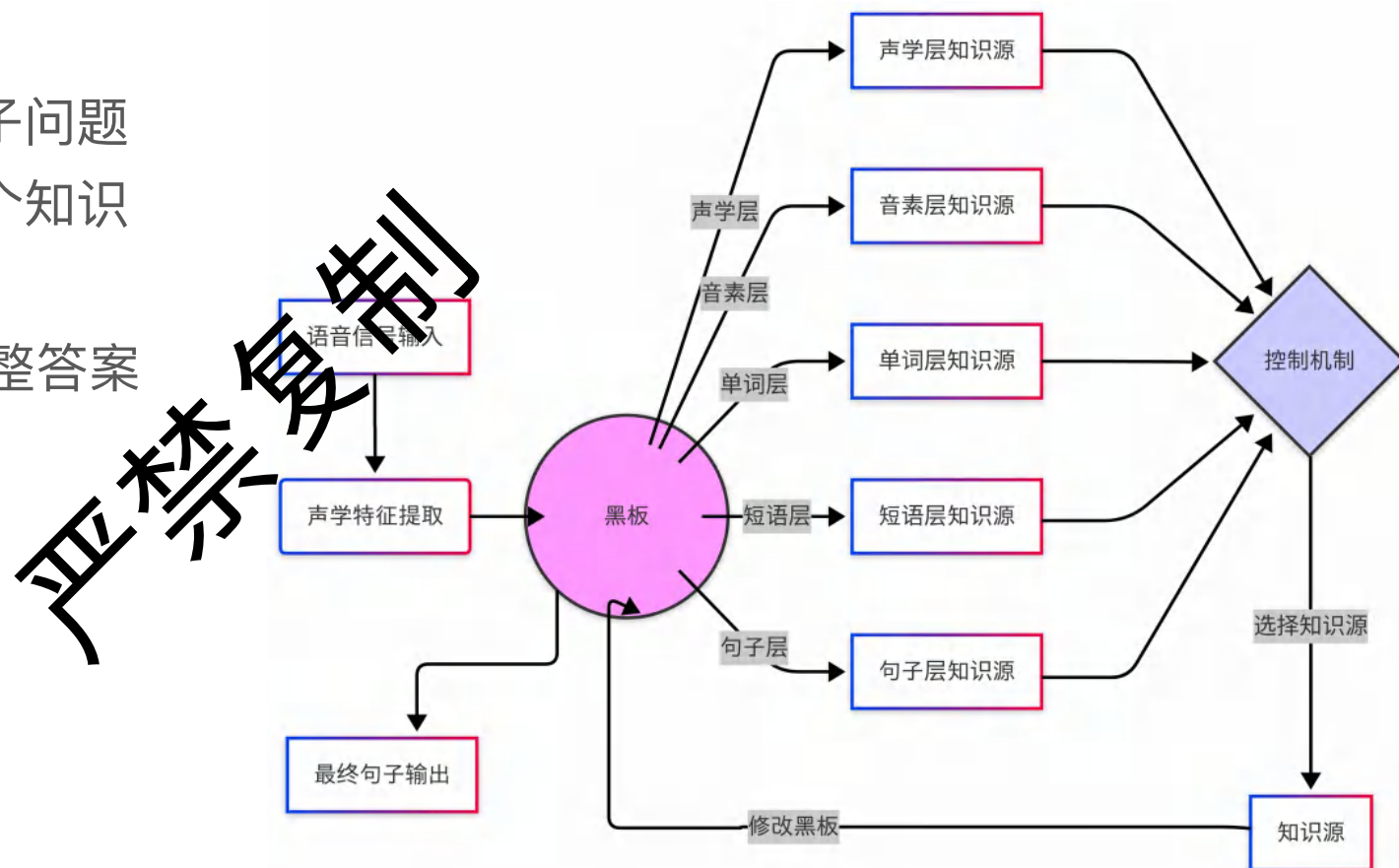
- 语音识别系统
 - HEARSAY-II
- 模式识别
- 天气预报
- 游戏
- 分子结构分析
 - Crystalis



Blackboard

- 约束

- 可分解：问题能够被分解成多个子问题
- 可解决：每个子问题都可以被一个知识源解决
- 可整合：Blackboard能够把不完整答案整合起来



Blackboard

1

优势

1. 实验性质
 1. 解决开放性的问题
 2. 策略变更很容易
2. 复用性高
 1. 知识源都是可复用的
3. 容错性

2

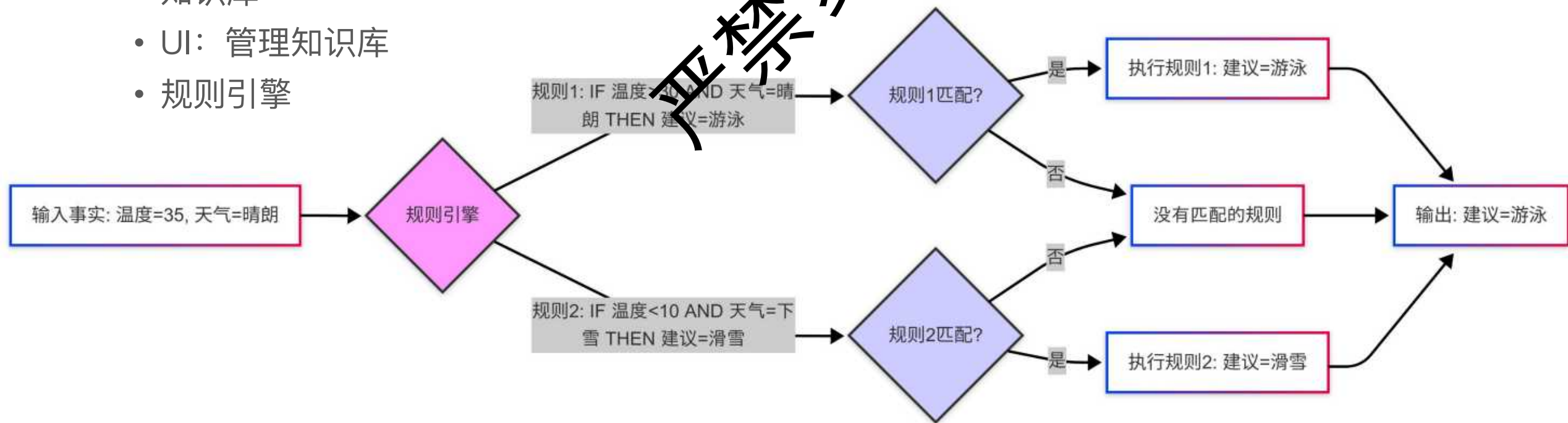
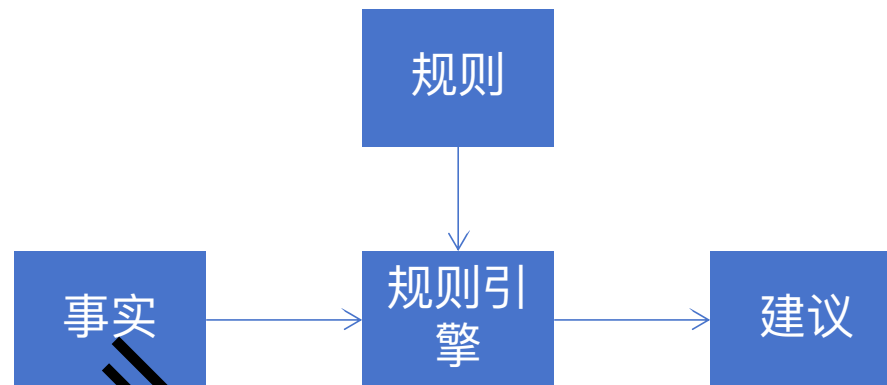
挑战

1. 调试困难
 1. 不能保证最终结果是正确的
 2. 控制策略难以控制
2. 性能
 1. 需要设计好各个数据源的协同机制
3. 研发难度高
 1. 并行执行各个ks
 2. 必须同步控制多个ks访问

Blackboard

基于规则的系统

- 共享内存的变体
 - 把共享内存变成共享知识库
 - 包含规则+事实
- 要素
 - 知识库
 - UI：管理知识库
 - 规则引擎



基于规则的系统-应用

- 专家系统
- 产生式系统：
 - 规则库，动态工作记忆，推理机
 - 不断利用规则来改变记忆，穷尽各种可能性
- JAVA的规则引擎
 - JRules, Drools, JESS
- 基于规则的编程语言
 - Prolog
- 业务规则管理系统

```
% 事实：定义谁是谁的父母
parent(F, mary). % John 是 Mary 的父母
parent(F, peter). % John 是 Peter 的父母
parent(M, mary). % Susan 是 Mary 的父母
parent(M, peter). % Susan 是 Peter 的父母
parent(mary, k1). % Mary 是 Ann 的父母
parent(peter, k2). % Peter 是 Tom 的父母
```

```
% 规则：定义父亲关系
father(X, Y) :- parent(X, Y), male(X).
% 规则：定义母亲关系
mother(X, Y) :- parent(X, Y), female(X).
% 规则：定义祖父母关系
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
% 规则：定义兄弟姐妹关系
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

```
% 事实：定义性别
male(john).
male(peter).
male(tom).
female(susan).
female(mary).
female(ann).
```

问题：谁是k1的爷爷？
回答：F

基于规则的系统

- 约束
 - 知识库需要保存完整的领域知识
 - 修改知识必须通过管理工具（不能直接update）
 - 格式规范：IF 前提条件 THEN 得到什么后果

严禁复制

Rule based systems

- 优势
 - 可维护性
 - 修改规则很容易
 - 关注点分离
 - 技术人员只关心算法
 - 业务人员只关心规则
 - 可复用性
- 挑战
 - 调试困难（定位出错的规则）
 - 性能问题
 - 收集事实
 - 规则的创建和维护
 - introspection（反省）
 - 自动化学习规则（机器学习）
 - 运行是规则变更（促销）

严禁复制

5-Invocation远程调用模式

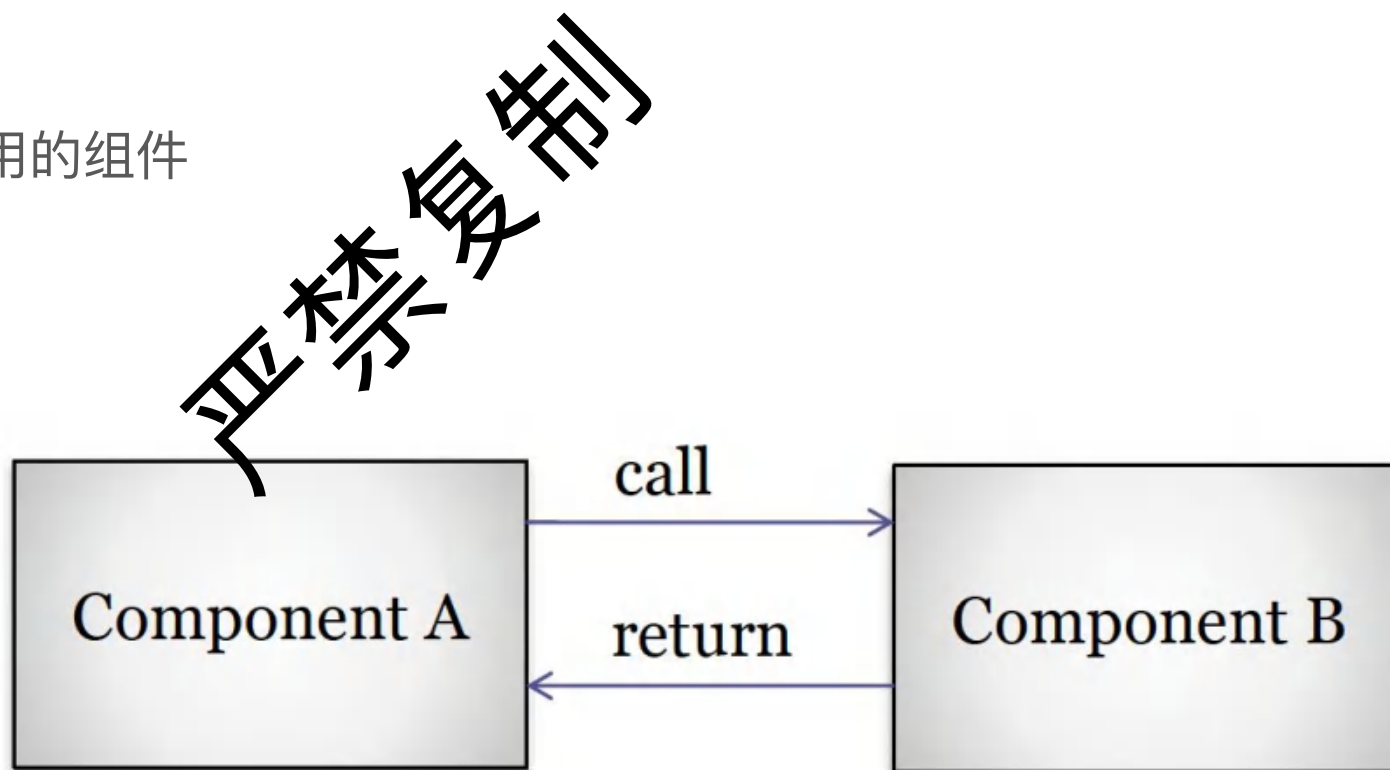
- call-return
- client-server

系统拆分成两个部分，独立部署，一部分负责为用户提供服务，另一部分负责底层服务。

严禁复制

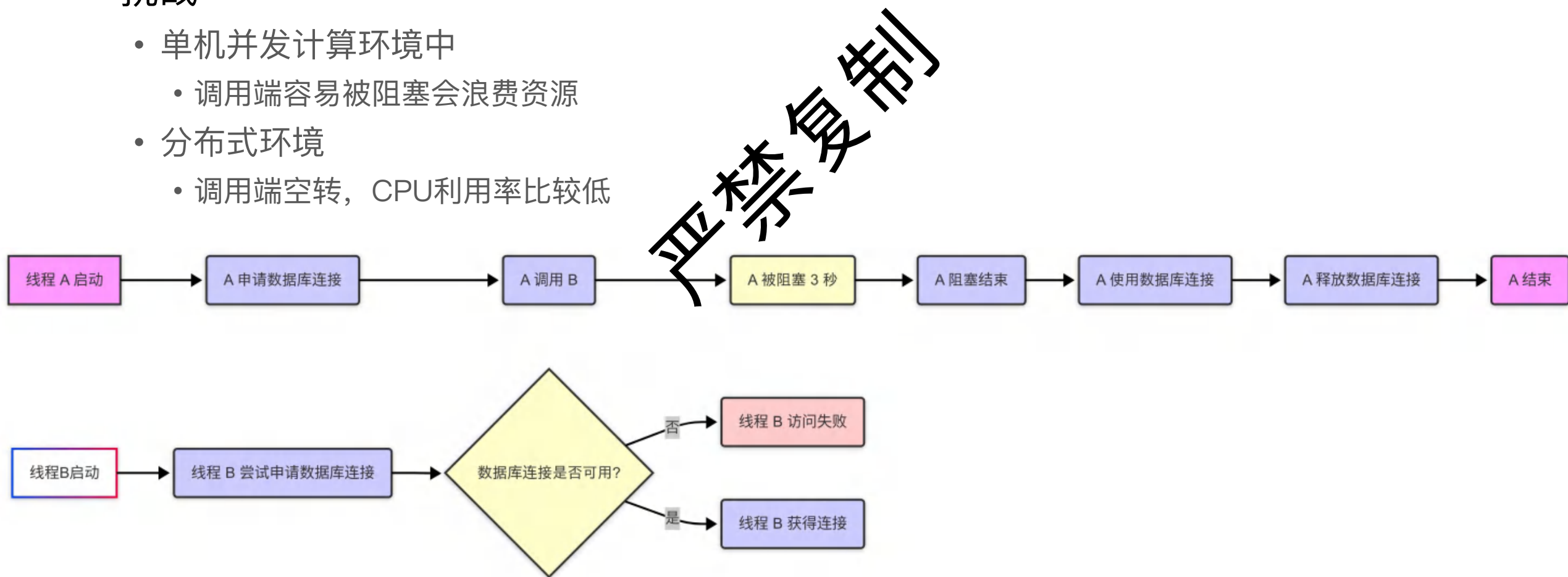
call-return

- 组件A发起调用，等待返回
 - 银行取款机上取钱
- 要素
 - 发起调用的组件，接受调用的组件
- 约束
 - 同步通信



Call-return

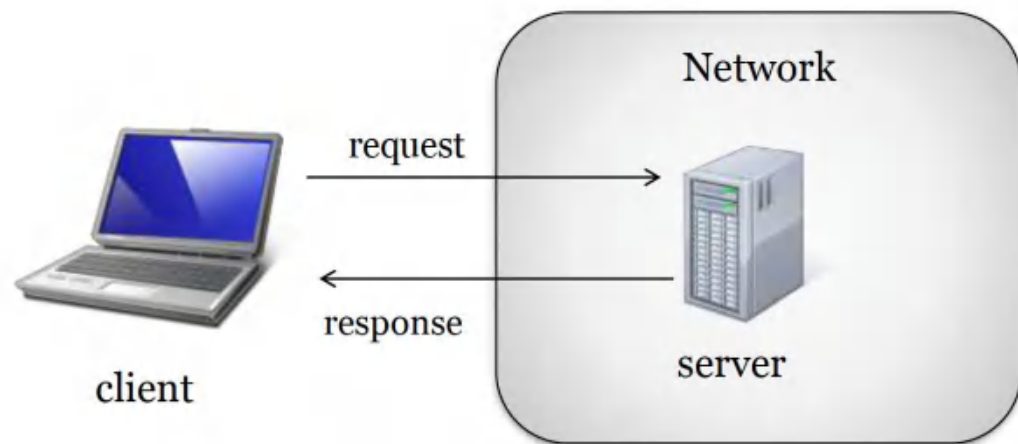
- 优势：容易实现
- 挑战
 - 单机并发计算环境中
 - 调用端容易被阻塞会浪费资源
 - 分布式环境
 - 调用端空转，CPU利用率比较低



Client-Server

- layer的变体
 - 物理分隔的2层 (2-tier)
- 功能被分开到几个server中
- Clients连接到多个不同的server
 - 通过接口发送request接收response

严禁复制



Client-Server

- 要素：
 - Server：通过协议提供服务
 - Client：发起请求，并处理回答内容
 - 网络通信协议：client和server之间的沟通协议
- 约束
 - clients必须和servers进行沟通
 - 获取数据的唯一通道
 - 多个client之间互相独立
 - servers对client知道很少（可能是伪造请求）
 - 网络协议提供通信保证（可靠性，顺序性，安全加密）

禁止复制

Client-Server

- 优势

- 分布式：多个server分布式运行
- 低耦合：
 - client和server职责分开
 - client和server独立开发
- 可扩展
- 可靠性
 - 所有client都可以使用
 - 部分server崩溃不影响整体

- 挑战

- 每个server都可能是单点（DDoS攻击）
- 性能
 - 依赖于系统和网络
 - 其它团队运行的server可能导致性能问题

严禁复制

Client-Server的变体

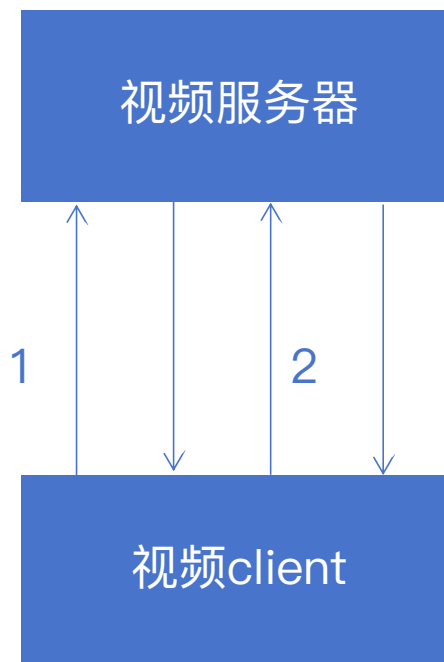
- 无状态
- Server复制
- Server with cache

严禁复制

无状态CS模式

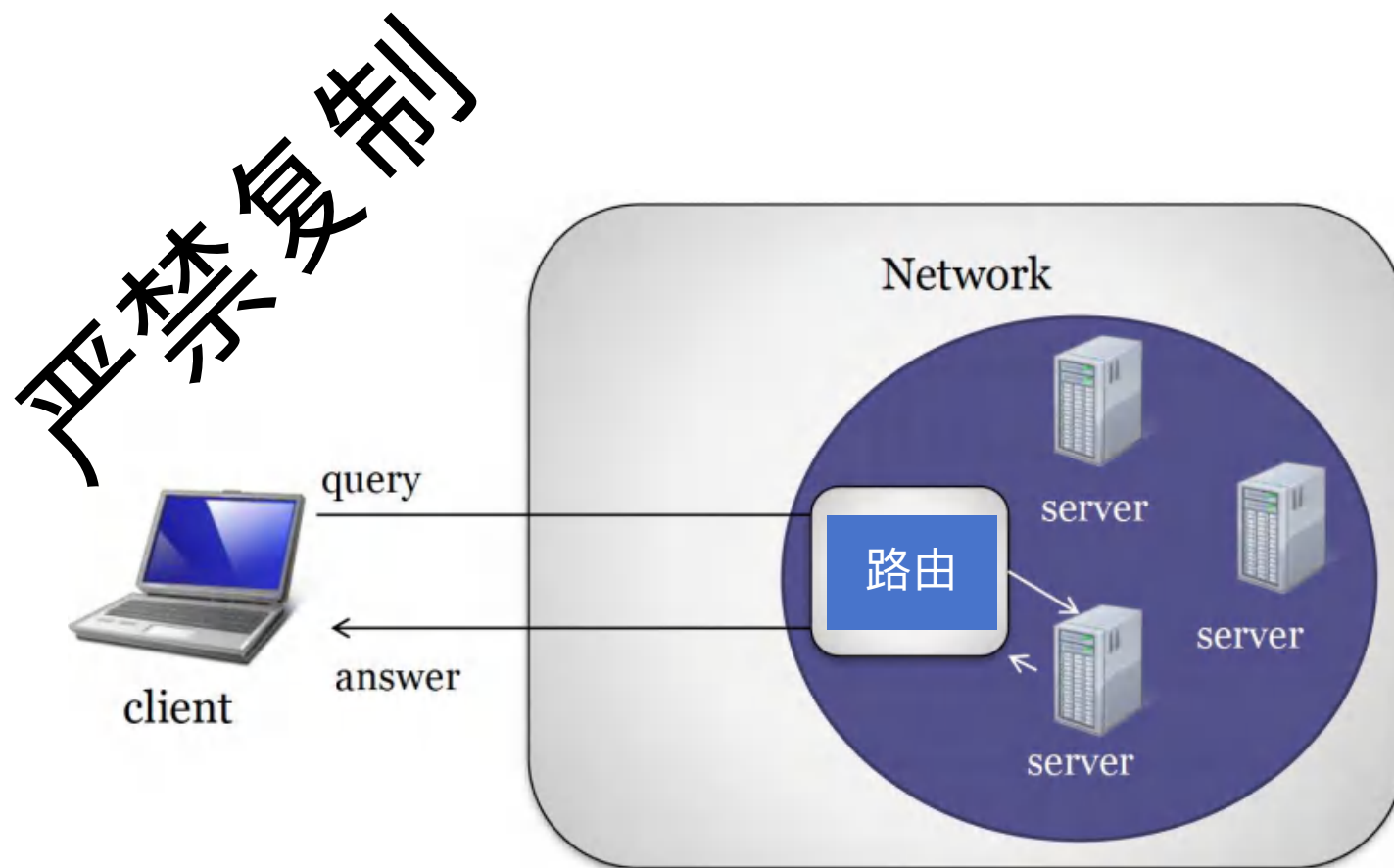
- 特点
 - server不保存client的状态
 - 同一个query总是返回同一个answer
- 优势
 - scalable
- 挑战
 - 应用状态的管理
 - Client必须记住request
 - 管理多个requests之间的信息

严禁复制



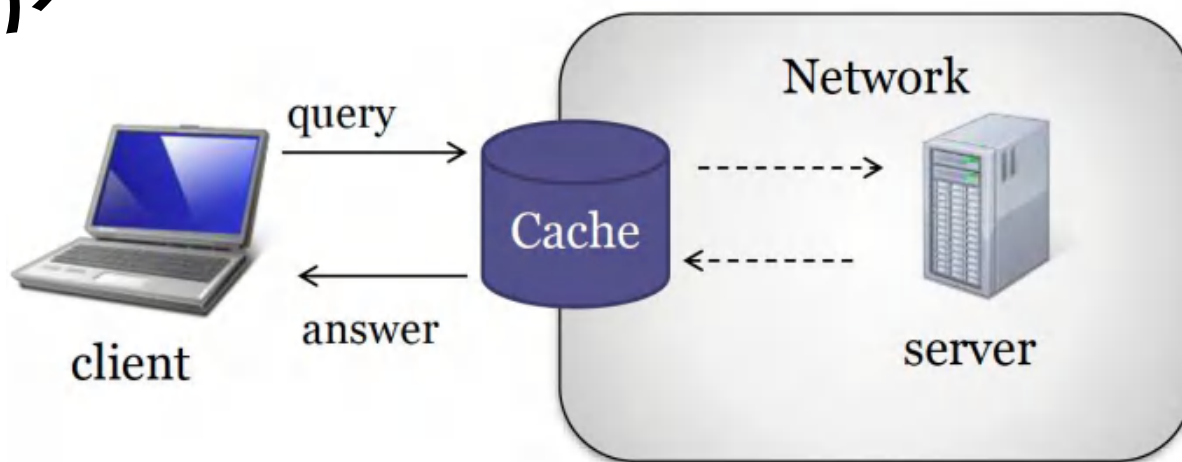
Replicated 多备份服务器

- 要求
 - 多个服务器提供同样的服务
 - 在client看起来只有一台server
- 优势
 - 支持更多请求
 - 更快
 - 容错
- 挑战
 - 多个server的一致性
 - 多个server之间的数据同步



有cache的Client-server

- Cache是client和server的中间人
 - 保持server之前的answer的copy
 - 当query第二次到达时，直接返回之前的answer
- 要素
 - 中间缓存节点
- 约束
 - 同样的query（字面上）必然是同样的answer
 - answer的管理策略



有缓存的CS架构

- 优势

- 降低网络负载
 - 很多重复的请求可以存在cache中
- 减少回答时间
 - 使用cache能更快的产生答案

- 挑战

- 难以配置
- cache逾期策略的配置
- 一些领域并不适用
 - 高保真领域例如实时系统

严禁复制

THANK YOU

北京交通大学软件学院

严禁复制

